

Alma Mater Studiorum Università di Bologna
Archivio istituzionale della ricerca

A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference

This is the final peer-reviewed author's accepted manuscript (postprint) of the following publication:

Published Version:

Le Gallo, M., Khaddam-Aljameh, R., Stanisavljevic, M., Vasilopoulos, A., Kersting, B., Dazzi, M., et al. (2023). A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. NATURE ELECTRONICS, 6(9), 680-693 [10.1038/s41928-023-01010-1].

Availability:

This version is available at: <https://hdl.handle.net/11585/961655> since: 2024-02-26

Published:

DOI: <http://doi.org/10.1038/s41928-023-01010-1>

Terms of use:

Some rights reserved. The terms and conditions for the reuse of this version of the manuscript are specified in the publishing policy. For all terms of use and more information see the publisher's website.

This item was downloaded from IRIS Università di Bologna (<https://cris.unibo.it/>).
When citing, please refer to the published version.

(Article begins on next page)

A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference

Manuel Le Gallo,^{1,2, a)} Riduan Khaddam-Aljameh,^{1,2} Milos Stanisavljevic,^{1,2} Athanasios Vasilopoulos,² Benedikt Kersting,² Martino Dazzi,² Geethan Karunaratne,² Matthias Brändli,² Abhairaj Singh,² Julian Büchel,² Xavier Timoneda,² Vinay Joshi,² Urs Egger,² Angelo Garofalo,² Anastasios Petropoulos,³ Theodore Anthonakopoulos,³ Kevin Brew,⁴ Samuel Choi,⁴ Injo Ok,⁴ Timothy Phillip,⁴ Victor Chan,⁴ Claire Silvestre,⁴ Ishtiaq Ahsan,⁴ Nicole Saulnier,⁴ Vijay Narayanan,⁵ Pier Andrea Francese,² Evangelos Eleftheriou,² and Abu Sebastian^{2, b)}

¹⁾These authors contributed equally

²⁾IBM Research - Europe, 8803 Rüschlikon, Switzerland

³⁾University of Patras, 26504 Rio Achaia, Greece

⁴⁾IBM Research - Albany, NY 12203, USA

⁵⁾IBM Research - Yorktown Heights, NY 10598, USA

(Dated: 1 November 2022)

Analog in-memory computing (AIMC) with resistive memory devices promises to reduce the latency and energy consumption of deep neural network inference tasks by performing computations directly within memory. However, to achieve end-to-end improvements, AIMC must be combined with on-chip digital operations and communication to realize a full inference workload entirely on-chip. Here, we present a multi-core AIMC chip designed and fabricated in 14-nm complementary metal–oxide–semiconductor (CMOS) technology with backend-integrated phase-change memory (PCM). The fully-integrated chip features 64 256x256 AIMC cores interconnected via an on-chip communication network. It also implements the digital activation functions and processing involved in ResNet convolutional neural networks and long short-term memory (LSTM) networks. We demonstrate near software-equivalent inference accuracy with ResNet and LSTM networks while implementing all the computations associated with the weight layers and the activation functions on-chip. The chip can achieve a maximal throughput of 63.1 TOPS at an energy efficiency of 9.76 TOPS W⁻¹ for 8-bit input/output matrix-vector multiplications.

The need to repeatedly shuttle around synaptic weight values from memory to processing units has been a key source of energy inefficiency associated with hardware implementation of artificial neural networks. Analog in-memory computing (AIMC) with spatially instantiated synaptic weights is arguably the most promising approach to overcome this challenge. Early works on performing neural network inference with AIMC showed promising accuracy results in mixed hardware/software implementations, where functionalities such as digital-to-analog and analog-to-digital conversions, activation functions, and other necessary digital operations were implemented with off-chip software or hardware^{1,2}. Since then, various single- or few-core chips using SRAM³, Flash⁴, RRAM^{5,6}, PCM⁷, and MRAM⁸ memory technologies have been fabricated with fully-integrated data conversions and sometimes activation functions^{4,5,7}, demonstrating a competitive energy efficiency of ≥ 10 TOPS W⁻¹ for reduced-precision matrix-vector multiplication (MVM). In such works, only small networks ($\lesssim 100k$ weights) on simple benchmark tasks such as MNIST digit recognition have been implemented to fit entirely on the core⁴⁻⁷, whereas larger networks have been emulated by programming each layer one at a time onto the same core^{3,8}. Recently, multi-core chips supporting larger networks ($> 1M$ weights) have demonstrated promising inference accuracy results on more difficult benchmarks, such as image recognition on the CIFAR dataset, and high MVM energy efficiency⁹⁻¹⁴.

Despite those advances, significant challenges remain to build a complete AIMC accelerator to demonstrate end-to-end inference tasks with accuracy and efficiency that outperform the already existing digital accelerators. Because of the rather large size of the multi-bit SRAM plus switched-capacitor unit-cells, the existing SRAM-based implementations need off-chip weight buffers to hold network weights, which are then partially transferred to the AIMC cores because an entire network cannot fit fully on-chip¹³. The current implementations are not dense enough to hold $> 10MB$ of weight data at reasonable chip area, limiting their efficiency gains over digital approaches to rather small networks that fit entirely on-chip¹⁵. Another limitation is the volatility of the SRAM cells, which prevent the on-chip weights to be retained throughout power cycling of the accelerator.

The potential to achieve smaller unit-cell sizes with resistive memory technologies could make it possible to hold such large networks entirely on-chip in a non-volatile way and eliminate the overhead of off-chip weight data access. In this context, an AIMC accelerator based on such devices becomes a natural choice. Both digital^{9,11} (1 device corresponds to 1 weight bit) and analog (1 device encodes an entire weight)¹⁰ storage of weights in those devices have been explored in the recent AIMC chips. Digital storage offers high (almost digital-like) accuracy, but the throughput is limited due to small number of rows that can be operated at a time (sometimes as low as 4)¹¹ without degrading the signal quality. This approach also limits the achievable

^{a)}Electronic mail: anu@zurich.ibm.com

^{b)}Electronic mail: ase@zurich.ibm.com

weight density because several devices are needed to encode a weight. Analog weight storage offers high weight density and a larger number of rows (up to 256)¹⁰ have been possible to operate simultaneously with this approach. However, it suffers from accuracy degradation because of the noisy analog weights and higher latency due to the need of slow high resolution analog-to-digital converters (ADCs) that often have to be multiplexed across columns. Moreover, extensive network-specific chip tuning, such as per-network recalibration of the chip to mitigate non-idealities of the devices and peripheral circuitry, and chip-in-the-loop finetuning of weights through backpropagation, has so far been necessary to achieve acceptable inference accuracy with this approach^{1,10}. Finally, at the system level, none of the current AIMC chips based on resistive memory support all the computations involved in processing convolutional and LSTM layers.

To address those challenges, we present Hermes: a 64-core AIMC chip based on back-end integrated PCM in a 14-nm CMOS process. The chip delivers simultaneously four key advances over the previous implementations discussed above. (i) High weight capacity is enabled through the analog storage capability of PCM, using only 4 PCM devices to encode a weight. (ii) Low MVM latency, high parallelism, and high throughput are achieved by using compact current-controlled oscillator ADCs placed on each unit-cell row to perform fully-parallel MVMs. (iii) High MVM and inference accuracy are achieved without application-wise chip re-tuning, thanks to proper on-chip ADC calibration circuits that remove unwanted nonlinear response. (iv) Finally, all non-MVM computations involved in processing convolutional and LSTM layers are supported using dedicated high-precision digital units. We report measured inference results at near software-equivalent accuracy on CIFAR-10 image recognition, Penn Treebank (PTB) character prediction, and image caption generation tasks with all convolutional and LSTM layer computations performed on-chip. The measured MVM throughput per area of the chip is $> 15\times$ higher than comparable multi-core resistive memory AIMC chips, while achieving the highest accuracy on the CIFAR-10 benchmark.

I. CHIP ARCHITECTURE

Fig. 1a shows a snapshot of the chip inside the electronic design automation (EDA) tool. The chip has a square dimension of $12\text{ mm} \times 12\text{ mm}$. Its 64 cores, each of size $1.2\text{ mm} \times 1.16\text{ mm}$, are placed in eight rows and eight columns. This subdivided structure is detailed in the schematic illustration in Fig. 1b. Each core contains a PCM crossbar array capable of storing a 256×256 weight matrix and performing an analog MVM using the input activations provided from outside of the core. Hence, up to 4,194,304 weights can be stored on-chip for performing in-memory MVMs.

At the center of the core grid, between rows four and five, a row of eight global digital processing units (GDPUs) is placed. They provide the additional digital post-processing capabilities needed when running LSTM networks. The high-frequency clock signals for the digital circuitry in the cores and GDPUs are received at the center of the chip and are distributed in a balanced tree across the chip.

Finally, the core outputs and GGPU inputs are interconnected by a grid of horizontal and vertical digital communication links (see Fig. 1b), yielding a DNN acceleration architecture where weight elements are stationary, and only the activation vectors are transmitted. There are 418 physical communication links in total, which together implement a 5 Parallel Prism communication fabric topology¹⁶ (see Extended Data Fig. 1 for a map of all possible link connections). A summary of the chip specifications can be found in Extended Table I.

II. THE COMPUTATIONAL MEMORY CORE

Fig. 1c depicts the architecture of a single Hermes core and its various components. The design is based on the fully-integrated standalone PCM-based single-core chip presented in Ref. 7, with the addition of the link controller for sending and transmitting of input activations between different cores. At the center of the core, there is a 256×256 -sized crossbar array of PCM-based unit-cells. As shown in Ext. Data Fig. 2a, four PCM devices are used per unit-cell: two each for each polarity, such that positive and negative weights can be represented^{17,18}. The devices in a conductance pair are weighted with equal significance.

To program the individual conductances, a diagonal decoding scheme¹⁸ is employed to select 256 different devices, one per row and column. The whole procedure starts with the dedicated per-core programming finite-state machine (FSM) instructing the diagonal selection decoder to enable one diagonal of cells that contains the devices that are to be programmed (see Ext. Data Fig. 2b). Subsequently, these selected devices can be programmed by the current-steering digital-to-analog converter (DAC) based programming units located on top of the PCM array. The DAC units are capable of producing all required programming pulse shapes for the PCM devices based on the instruction provided by the FSM. This includes high current RESET pulses, triangular-shaped SET pulses, and programming pulses with variable current amplitude and time duration.

Once all programming operations are concluded, the core can be used to perform analog MVMs. To this end, the input modulator applies pulse-width modulated (PWM) read voltage pulses to the PCM array, which are digitized by an array of 256 time-based current-ADCs. These ADCs flank the PCM-array from the left and right side, such that all unit-cell rows are covered by a dedicated ADC. This is achieved by matching the ADC height to that of two unit-cells, which results in a skewed aspect ratio, as is shown in the layout in Ext. Data Fig. 2c. By choosing a current-based ADC topology, area-intensive and noise-prone

current-to-voltage conversion steps can be avoided. Furthermore, by opting for a time-based architecture, a highly digital design is obtained in which the output precision can be adjusted by changing the conversion time.

To counteract manufacturing mismatch and other non-idealities, the ADC includes an internal resistor ladder-based voltage DAC that allows correcting the read-voltage offset. Furthermore, the internal current mirror, which drives the attached current-controlled oscillator (CCO) unit, contains trimming registers that allow matching the gain between the different ADCs per core. Thus, after performing an initial calibration procedure, the configuration parameters are stored on-chip such that the ADCs exhibit tightly distributed transfer curves. Note that this calibration is independent of the deployed application and does not have to be redone for different weight sets or networks.

Ext. Data Fig. 3a illustrates one row of the computational memory during an analog MVM. The per-row ADC regulates the BL to a fixed common-mode voltage, such that the top electrodes of all PCM devices in the 256 unit-cells are on the same potential. Their respective SL potentials, however, are connected to one of three potentials based on the respective input value, i.e., the positive read voltage V^+ to subtract current, the negative read voltage V^- when adding current, or high impedance when no current from the cell is to be added. Thus, all combinations of products between a positive or negative weight and a positive or negative input can be performed in the analog domain in a single modulation, as shown in Ext. Data Fig. 3b. In addition, a 4-phase mode is supported that is shown in Ext. Data Fig. 3c. Therein, the four combination of $\pm$ weights and $\pm$ inputs are performed in four separate operations while only using the negative read voltage V^- . As a result, a higher accuracy of the measured current is achieved, as all PCM devices are read in the same polarity by the current-based ADC.

The ADC delivers the final results as two 12-bit unsigned integers, one for the positive and one for the negative current. By using tri-state buffers on a 24-bit wide bus, the results are transmitted from each ADC to the local digital processing unit (LDPU) for post-processing. In the first step, the results are transferred to a register array, such that the operation becomes fully pipelined. Consequently, the initial latency of the LDPU does not affect its throughput (one data point per clock cycle). Next, the results are transferred one by one to the ADC-convert-and-scale block, which first converts the 12-bit integers to single-precision floating-point format (FP16) and then employs two fused multiply-add (FMA) units to remove gain and offset errors.

Two ADC-convert-and-scale blocks produce outputs every second clock cycle and feed them one after the other to the activation function block through a multiplexer unit. There, affine scaling per channel, as well as an optional ReLU operation, are applied. The intermediate value is then combined with an 8-bit signed integer (INT8) input coming from the neighboring cores, or from off-chip, through the link controller receiver (RX). This value is first converted into FP16 format and then scaled to the optimal range. Finally, the output undergoes another optional ReLU operation before conversion into INT8, ready to be shipped by the transmitter (TX) of the link controller. The existence of both ReLU operations allows the possibility to apply the activation both before and after adding the RX link input. The activation block is fully pipelined so that its latency does not affect the LDPU throughput.

The link controller implements a state machine that enables transmission and reception of data within at most six neighboring cores (receiving and transmitting cores are not necessarily the same, see Ext. Data Fig. 1). The core-to-core input-output links are implemented as eight parallel channels, each transferring one bit every clock cycle. The selection of the data source for the TX and of the data destination for the RX is implemented through a routing table, configured before runtime. Specifically, the transmitter-side link controller prepends a preamble to the payload being sent on the links. The preamble contains information to uniquely identify a routing path for a particular type of payload. If the receiving core is enabled to receive data from the transmitting core in the routing table, the link controller samples the incoming data. Furthermore, the link controller in the transmitting and receiving cores can select a portion of the payload according to another set of routing registers.

III. THE GLOBAL DIGITAL PROCESSING UNIT

The GPDU is responsible for the digital processing required in LSTM networks. The Hermes chip contains eight GPDU slices in total, each connected to the fourth row core above a GPDU slice. The cores in a column above a GPDU slice can aggregate their outputs in a cascaded fashion in the LDPUs of those cores. The final aggregated output produced by the LDPU of the fourth row core is then sent to its corresponding GPDU slice. Each slice takes up to 256 inputs serially (one in each clock cycle), out of which 64 inputs of input gate's (I), cell input (A), forget gate's (F), and output gate's (O) pre-activation vector. Those pre-activation vector inputs arrive in an alternating fashion from four neighbouring BLs. By mapping columns of the I, A, F, and O weight matrices in an interleaved fashion, a single element processor per GPDU slice (instead of 64) can be used to process the 256 inputs serially, which significantly reduces the number of resources needed in the GPDU. Each GPDU slice produces one output every four clock cycles for up to 64 outputs. It also contains cell state memory registers for up to 64 elements.

Fig. 1d depicts one GPDU slice architecture in detail. All computations inside the GPDU slice are performed in the FP16 format with inputs and outputs in INT8 format. Hyperbolic tangent activation functions are realized using a lookup table (LUT) method. First, a bank of 17 comparators is used to find the corresponding bin (out of 18) in which an input is contained. Then, the output value is interpolated in a fused-multiply-add (FMA) unit using the slope and offset read from the LUT for the determined bin. The output activations of the four gates are further processed by two multiply blocks, three FMA blocks, and one more hyperbolic tangent LUT block to compute the cell state and the hidden state output of the LSTM unit.

148 IV. MVM ACCURACY

149 We characterized all 64 cores of Hermes to assess the PCM yield and how accurately MVMs can be performed with them.
 150 Fig. 2a shows the SET and RESET distributions of the unit-cells of all 64 Hermes cores. The Hermes unit-cell comprises
 151 four PCM devices (two per polarity). Therefore, we define a unit-cell in the RESET state (G_{RESET}) when all PCM devices are
 152 programmed to RESET, and a unit-cell in the SET state (G_{SET}) when one PCM device is programmed to SET and the others to
 153 RESET. On 63 out of the 64 cores, more than 99% of the unit cells can be programmed to $|G_{\text{RESET}}|$ smaller than 5 ADC counts
 154 and G_{SET} larger than 50 ADC counts. The yield of the remaining outlier core is 98.4%. The high yield achieved on all cores
 155 implies that it is possible to program most of the unit-cells to a given conductance value via iterative programming, as long as it
 156 is within the range of their G_{RESET} and G_{SET} .

157 We studied two programming algorithms using either strictly one device (ODP) or up to two devices (TDP) to map a weight
 158 to a unit-cell conductance. A weight is written by programming the unit-cell devices that correspond to the weight polarity.
 159 Devices of the opposite polarity are RESET to near zero conductance. For a given unit-cell at row m and column n in a Hermes
 160 core, the corresponding weight W_{mn} is mapped to a target conductance value G_{mn} according to

$$G_{mn} = W_{mn} \cdot \frac{G_{\text{max}}}{W_{\text{max}}}, \quad (1)$$

161 where W_{max} is the maximum absolute weight value to be programmed in the Hermes core, and G_{max} is the maximum reliably
 162 programmable unit-cell conductance. For ODP, G_{max} should ideally not be larger than the least conductive SET device of the
 163 core. We define $G_{\text{max}} = 80$ ADC counts based on SET distributions of Fig. 2a, as the tenth percentile of the core with the least
 164 conductive SET states. For TDP, G_{max} should be set according to the least conductive unit-cell when both devices of positive or
 165 negative polarity are in SET state. Here, we choose $G_{\text{max}} = 160$ ADC counts.

166 We use a closed-loop iterative programming scheme in which the programming current is updated proportional to the dif-
 167 ference between the read conductance of unit-cell (m, n) and G_{mn} ¹⁹. In each iteration, the devices are read individually, and a
 168 programming pulse is applied to the unit-cells that did not yet converge within a pre-defined margin. For ODP, one out of the two
 169 unit-cell devices corresponding to the weight polarity receives the iterative programming pulses, and the other is RESET. For
 170 TDP, if G_{mn} exceeds G_{SET} of both devices, one is programmed to the SET state and the other receives the iterative programming
 171 pulses. Otherwise, the device with the highest G_{SET} receives the iterative programming pulses and the other is RESET (see
 172 Methods and Extended Data Fig. 4). Performing the device selection this way ensures that a maximal amount of devices are at
 173 RESET or SET, which are the least noisy states. Because of this, reduced programming errors are achieved compared with prior
 174 multi-device programming approaches that attempt to program each unit-cell device via iterative programming up to the ODP
 175 G_{max} ²⁰ (see Extended Data Fig. 4c).

176 To quantify the MVM accuracy achieved with ODP and TDP on the 64 Hermes cores, a uniformly distributed weight matrix
 177 W with 30% sparsity is programmed on each core. 2,048 input vectors x , uniformly distributed with 30% sparsity, are then
 178 sent to each core to perform the MVMs. Based on the measured MVM results y_{exp} , the actually programmed weights can be
 179 estimated as $\hat{W} = \text{argmin}_{\hat{W}} \|y_{\text{exp}} - \hat{W}x\|_2$. We then characterize the MVM error using the three following metrics:

$$\epsilon_{\text{total}} = y_{\text{exp}} - y_{\text{fp}} \quad (2)$$

$$\epsilon_{\text{linear}} = \hat{W}x - y_{\text{fp}} \quad (3)$$

$$\epsilon_{\text{residual}} = y_{\text{exp}} - \hat{W}x, \quad (4)$$

180 where $y_{\text{fp}} = Wx$ denotes the MVM result computed in double precision floating-point. ϵ_{total} is the total MVM error, which we
 181 decompose into a linear contribution ϵ_{linear} and a residual contribution $\epsilon_{\text{residual}}$. In a first approximation, ϵ_{linear} defines the error
 182 resulting from a wrong programming of the weights. $\epsilon_{\text{residual}}$ results from any residual error of the chip that cannot be cast as a
 183 weight error, such as PWM and ADC nonlinearities, instantaneous read noise, leakage current and IR drop.

184 The error histograms of the three metrics over all 64 cores are plotted in Fig. 2b for ODP and TDP. For ODP, ϵ_{total} is largely
 185 defined by ϵ_{linear} , and $\epsilon_{\text{residual}}$ is comparably negligible. When using TDP, ϵ_{linear} is significantly reduced compared with ODP,
 186 showing that the weight programming error is effectively lowered when using more devices per weight. In this case, the impact
 187 of $\epsilon_{\text{residual}}$ gets noticeable in the total error. We ascribe the wider spread of $\epsilon_{\text{residual}}$ for TDP to the fact that a larger range of the
 188 ADC transfer curve and particularly the more nonlinear high-current-range is sampled. Moreover, higher leakage from the PCM
 189 array than ODP is expected because more devices are programmed to non-RESET states, which also increases $\epsilon_{\text{residual}}$. Fig. 2c
 190 shows the weight error plotted as a function of target weight, confirming that TDP allows to reduce it for all nonzero weight
 191 values. We attribute this reduction to a higher signal-to-noise ratio achieved when using two devices instead of one²⁰.

192 For resistive memory technologies, it is also critical to study the degradation of the MVM accuracy over time. In PCM, the
 193 conductance values drift with time t according to $G(t) = G(t_0)(t/t_0)^{-\nu}$, where $G(t_0)$ is the conductance at t_0 and ν denotes the
 194 drift exponent. Drift can partially be mitigated by a global drift compensation procedure, i.e., an appropriate per-core rescaling
 195 the MVM result^{20,21}. However, since each programmed weight drifts with a slightly different rate (ν depends on the conductance
 196 state and varies from device to device), ϵ_{linear} and thus also ϵ_{total} increase over time as shown in Fig. 2d. The subtle reduction of

$\epsilon_{\text{residual}}$ can be ascribed to the sampling of a gradually smaller range of the ADC transfer curve. Compared with an equivalent digital engine having 8-bit input/output precision and N -bit weight precision, the precision achieved with ODP is close to 3-bit weights. With TDP, a precision between 3-bit and 4-bit weights is achieved.

V. DEEP NEURAL NETWORK INFERENCE DEMONSTRATIONS

Having established the MVM accuracy of Hermes, we now assess the inference accuracy when running different neural networks on the chip. We implemented three networks that cover all usable features of the chip: a ResNet-9 convolutional neural network (CNN) for CIFAR-10²² image classification, an LSTM network for predicting characters of the PTB dataset²³, and an LSTM network for generating captions of images in the Flickr8k dataset²⁴ (see Methods). Prior to being deployed on the chip, the networks are trained in a hardware-aware manner by injecting noise on the synaptic weights to improve their resilience to hardware nonidealities², using the publicly available IBM Analog Hardware Acceleration Kit²⁵ (see Methods for details about the training procedure). The trained weights are then mapped to conductance values to be programmed on the PCM unit-cells using Eqn. (1). G_{max} is determined on a per-core basis while constraining the BL current to not exceed the maximum supported by the ADC (see Methods). For the three networks, all operations involved in the processing of weight layers, including the MVMs, activation functions, batch normalization, LSTM cell-state computation, data aggregation when a layer is split onto multiple cores or for residual connections, bias addition, and all the necessary intra- and inter-layer affine scaling, are implemented on-chip. Apart from the max-pooling in ResNet-9 and the character/image/word embedding in the LSTM networks, which are not supported by the chip, no other computation is performed in software in the network executions.

The custom ResNet-9 network (1,866,536 synaptic weights and 10 biases) and its corresponding implementation on Hermes are shown in Fig. 3a and Fig. 3b, respectively. On-chip links are configured to aggregate MVM results in the LDPU of the cores implementing layers that cannot fit onto a single one (all layers except the first and the last). The batch normalization and ReLU activation for all convolution layers are performed in the LDPU of the core that receives the aggregated data. The residual connections are implemented by sending the residual data to the LDPU of the receiving core, and performing the addition inside the LDPU. For the last dense layer, the bias is programmed into the affine scale registers of the LDPU so that it gets added to the MVM result. The CIFAR-10 test accuracy results for ODP and TDP weight programming are shown in Fig. 3c. Hermes with TDP achieves an accuracy of 92.81%, which is less than 1% below the software baseline of 93.67%, whereas ODP achieves 92.23%. We also compare the experimental results with simulations that include the per-core weight error for this network extracted as in Fig. 2c, and the 8-bit quantization from the PWM and LDPUs (see Methods for details on the simulation model). The biggest accuracy drop from the software baseline comes from the weight error, and the digital quantization adds another 0.2 – 0.3% drop on top of it (see Fig. 3c). The accuracy obtained by simulating only those two effects nearly matches the experimental accuracy, which indicates that any additional nonlinearity from the peripheral circuitry of the chip does not have a significant impact compared with the weight and quantization errors.

The LSTM network for PTB character prediction (1,299,312 weights and 2,066 biases) is shown in Fig. 4a, and the corresponding Hermes implementation in Fig. 4b. Each character is first embedded into a 128-long random orthogonalized vector which is then input to the on-chip input gate. Then, MVMs from the input and hidden gates are summed vertically on-chip by the LDPUs of the 8 cores in domains 3 and 4, using vertical on-chip links to transmit the data. The data produced by the LDPUs of the 8 cores of domain 4 are then sent to the 8 corresponding GGPU slices using on-chip links. The GGPU then performs the sigmoid and tanh activations, the cell-state computation, as well as the necessary input/output data scaling. Finally, the output of the GGPU is sent to the dense layer which outputs the next predicted character in the sequence, as well as to the hidden gate as input for the character of the next timestep. The performance of the character prediction task is evaluated in bits per character (BPC), which is a measure of how well the model is able to predict samples from the true underlying probability distribution of the dataset (a lower BPC corresponds to a better model). The BPC results for ODP and TDP weight programming are shown in Fig. 4c as well as the corresponding simulation results. Hermes with TDP achieves a BPC less than 0.1 higher than the software baseline of 1.336 (adequate baseline on this benchmark for a network of this size²⁶), whereas the ODP BPC is only ~ 0.02 higher than TDP. Unlike for ResNet-9, the high per-core utilization of the LSTM network leads to a high enough current at the ADC input with ODP to achieve a good signal-to-noise ratio, therefore using TDP only marginally improves the BPC. Here again, the simulated accuracy with weight noise and quantization nearly matches the experimental accuracy.

The third network that we implemented is meant to assess the inference accuracy on a workload that uses all 64 cores of Hermes. We chose an image caption generation task implemented with a single LSTM unit and a dense layer (4,080,384 weights and 6,080 biases), similar to the PTB LSTM network described above (Fig. 5a). Here, at the initial timestep, the features of an input image extracted by a CNN (implemented in software) are fed to the on-chip input gate. As above, the MVMs from the input and hidden gates are summed on-chip and sent to the GGPU via on-chip links. The GGPU output is sent to the dense layer that produces the first word of the caption, which is stored in an off-chip caption buffer. Then, this word is embedded, sent to the input gate, and the GGPU output is sent to the hidden gate, in order to generate the next word of the caption (Fig. 5b). The latter process is run until the entire caption is generated, for at most 24 timesteps per image. The performance of the image caption generation task is evaluated using the BLEU- n score²⁷, which measures the match of word n -grams between a generated caption

and multiple reference captions. The BLEU scores measured over the full test dataset on Hermes are shown in Fig. 5c, along with three representative generated captions in Fig. 5d. The BLEU scores achieved by Hermes are essentially the same, within variation, as the software ones. It is observed that captions of some images differ between software and Hermes, sometimes one being better or worse than the other, as shown in the examples of Fig. 5d. However, when averaged over all images of the dataset, the Hermes BLEU scores are very similar to the software ones.

VI. PERFORMANCE

Table I summarizes the measured performance achieved by the Hermes chip for three use-cases: (i) the chip performing only MVMs at full utilization, (ii) the processing of one input to a deep layer of ResNet-9, and (iii) the processing of one timestep of the LSTM unit of the image captioning network (see Methods). Hermes achieves a peak MVM throughput of 63.1 TOPS at an energy efficiency of 9.76 TOPS/W and MVM area efficiency of 1.55 TOPS/mm² for the 1-phase read mode, when all 64 cores are fully utilized to perform MVMs in parallel. For the 4-phase read mode, the former metrics are approximately 4 times lower due to the $\sim 4\times$ higher latency. Even then, the raw MVM throughput of Hermes in 4-phase read mode is higher than the previous state-of-the-art AIMC chips based on RRAM, PCM and SRAM, and is comparable to the nor-Flash chip presented in Ref. 12, while achieving the highest accuracy on the CIFAR-10 benchmark (see Extended Data Table II). Although the SRAM-based accelerator presented in Ref. 13 shows a higher energy efficiency and throughput density, it requires the use of an off-chip weight buffer to store the total weight data of a network, leading to additional overhead during inference execution. In contrast, our implementation enables to store a model entirely on-chip, therefore reducing external memory accesses during execution as well as being able to retain the weight data throughout power cycling.

Finally, regarding the efficiency of the neural network layers including the digital operations, a single input to the ResNet-9 layer is processed in 1.52 μ s and consumes 1.51 μ J, whereas a single timestep of the LSTM unit is executed in 1.43 μ s while consuming 5.24 μ J. Reducing the precision of the digital computing from FP16 to lower integer precision, such as INT8, will allow in the future to parallelize the digital computations across ADC outputs to reduce the latency gap with the MVM-only latency.

VII. CONCLUSIONS

We have presented the Hermes chip, a 64-core AIMC chip based on 14-nm CMOS with backend-integrated PCM that can be used for deep neural network inference tasks. The chip can implement $> 4M$ weights split across 256x256 cores, perform MVMs through AIMC, and support all non-MVM computations involved in convolutional and LSTM layers via dedicated digital units. The chip achieves a peak MVM throughput of 16.1 – 63.1 TOPS at an energy efficiency of 2.48 – 9.76 TOPS/W in 4-phase (high-precision) – 1-phase (low-precision) read mode for 8-bit input/output MVMs. The measured MVM throughput per area of 400 GOPS/mm² in 4-phase read mode is $> 15\times$ higher than previous multi-core AIMC chips based on resistive memory, while achieving the highest CIFAR-10 accuracy of 92.81% among them. The next challenge for multi-core AIMC chips will be to enable the layer-to-layer activation transfers through the communication links, supporting the intermediate activation storage in local SRAM with appropriate appending and reorganization, in order to run a fully pipelined end-to-end inference workload on-chip. Further improvements in weight density will also be highly desirable for AIMC accelerators to be a strong competitor to existing digital solutions, which could be foreseen by integrating the PCM devices closer to the transistor-level at a denser pitch or via 3D stacking of memory layers.

288 METHODS

289 Chip and PCM device fabrication.

290 Our experimental results are demonstrated on chips from 300 mm wafers fabricated with PCM inserted in a 14 nm back-end-
 291 of-line (BEOL) at the IBM AI Hardware research center in Albany, NY. The BEOL fabrication including PCM mushroom cell
 292 formation is done on top of the foundry-sourced front-end wafers. The PCM mushroom cell is comprised of a ring heater for
 293 the bottom electrode and a $\text{Ge}_2\text{Sb}_2\text{Te}_5$ and top electrode film stack which is subtractively patterned to form the mushroom top.
 294 Wafers are characterized through several different in-line electrical tests to assess their quality before identifying wafers and die
 295 to send for dicing and packaging. Testing is done on discrete single structures embedded in the BEOL and also done on array
 296 diagnostic monitors (ADM) with functional peripheral circuitry.

297 Experimental platform.

298 The experimental platform is built around a single Hermes chip packaged in a 1,525 pin ball grid array (BGA). Through
 299 controlled collapse chip connection (C4), the chip can be mounted onto an interposer printed circuit board, which is placed on a
 300 socket for testing on a dedicated test board. For every core, 72 dedicated C4 pads provide analog and digital supply voltages in
 301 addition to a small number of pads used to implement a serial communication interface to control the core. The supply voltages
 302 of the cores in each row are grouped on the interposer. The resulting partition into eight power domains reduces the required
 303 number of components on the test board while maintaining a high degree of redundancy and resilience against faulty cores, for
 304 example due to manufacturing failures.

305 The Hermes chip is interfaced to a hardware platform comprising one FPGA module and a base board. The base board
 306 provides the chip socket, chip cooling, multiple power supplies per power domain as well as the voltage and current reference
 307 sources for the chip. A Trenz TE0808-04 module with Xilinx Zynq UltraScale+ field-programmable gate array (FPGA) is used
 308 to implement overall system control and data management as well as the interface with the Hermes chip. The experimental
 309 platform is operated through Ethernet from a host computer, and a Python environment is used to coordinate the experiments.
 310 The base board provides current measurement for all the Hermes chip supply rails by using high-side current sensing before the
 311 voltage regulators, allowing averaged current measurements with voltmeters.

312 Weight programming.

313 The gate signals of the unit-cells are connected in a diagonal fashion. The goal of this is to allow as many devices to be
 314 programmed in parallel as possible without causing an excessive current flow on the bitlines (BLs) and source lines (SLs).
 315 During programming, only one gate signal is enabled, as depicted in Extended Data Fig. 4a. These gate selection signals are
 316 controlled by the diagonal selection module.

317 The programming circuit controls the application of programming current profiles to the SLs of the crossbar array. There are
 318 a total of 32 parallel digital-to-analog converters (IDACs) that can apply digitally-controlled current values to the SLs. Each
 319 IDAC is assigned to 8 consecutive SLs of either negative or positive polarity (16 in total). A programming finite-state-machine
 320 is in charge of controlling these IDACs from the configuration values written to its registers, as well as selecting the active SLs
 321 of the PCM devices to program.

322 To program a set of devices, a single diagonal must be selected before the IDACs are activated. This way, current can be
 323 applied to all SLs in parallel without causing any current congestion in the chip. In practice only 32 SLs are programmed in
 324 parallel due to area and power constraints. Therefore, it takes 8 programming cycles to completely program one diagonal of
 325 devices for a particular polarity and device ID (A or B). Both square current pulses (RESET) and current pulses with a trailing
 326 edge (SET) are supported with configurable pulse widths and amplitudes. In the experiments presented here, we use a RESET
 327 pulse width of 125 ns with amplitude of $700\text{ }\mu\text{A}$, and a SET pulse width of 250 ns with trailing edge of 50 ns and amplitude of
 328 $125\text{ }\mu\text{A}$.

329 When programming a weight onto a unit-cell, all 4 PCM devices are first RESET, then the devices corresponding to the weight
 330 polarity are SET. For ODP, the SET pulses are sent only to the A device and for TDP to both A and B devices. Thereafter, the
 331 device which will receive the subsequent iterative programming pulses is selected. For ODP, device A is always selected. For
 332 TDP, if the target conductance value $|G_{mn}|$ exceeds G_{SET} of both A and B devices, the device with the lowest G_{SET} is selected
 333 for programming and the other is left as-is. Otherwise, the device with the highest G_{SET} is selected for programming and the
 334 other is RESET. Extended Data Fig. 4b shows a diagram of the TDP algorithm.

335 Iterative programming involving a sequence of program-and-verify steps is then used to program the selected devices to
 336 the desired conductance values.¹⁹ After each programming pulse, a verify step is performed, and the value of the unit-cell
 337 conductance programmed in the preceding iteration is read. The read is performed by applying long PWM pulses of 0.2 V

amplitude with 512 ns width and 256 ns precharge time to the selected diagonal, to ensure reliable read of the small individual conductance values by the ADCs. The programming current applied to the PCM device in the subsequent iteration is adapted according to the value of the error between the target level and the read value of the unit-cell conductance. The programming pulses are square pulses of 125 ns width and amplitude varying between 125 μ A and 700 μ A. The programming sequence ends when the error between the target conductance and the programmed conductance of the unit-cell is smaller than a margin of 5 ADC counts or when the maximum number of iterations (30) has been reached.

ResNet-9 on CIFAR-10 training.

The CIFAR-10 dataset²² is a well-known image classification benchmark that comprises a total of 60,000 RGB images belonging to one of 10 classes. The dataset is split into a training (50,000 images) and test (10,000) set. Each image has a size of 32×32 .

We use ResNet-9²⁸ since it is one of the most commonly used architectures for image classification. The architecture comprises 8 convolutional layers (all followed by a batch norm and ReLU activation layer) which can be attributed to two main blocks. Each block has two convolutional layers and a residual layer that contains another two convolutional layers. The number of filters for each convolutional layer in order of appearance is [56, 112, 112, 112, 224, 224, 224, 224]. This network yields a total of 1,869,122 trainable parameters.

We first train a model without any hardware-aware training for $N = 300$ epochs and later, as part of the hardware-aware training, finetune the pretrained model for additional $N = 200$ epochs. Note that, both the training phases are fully executed off-chip in software. The batch size is set to $B = 128$. In both phases of the training, we use SGD where the initial learning rate $\eta = 0.05$ is gradually reduced using a cosine annealing scheduler with $T_{\max} = 200$. In order to improve performance, we randomly crop the training images to a shape of 32×32 (padding=4) and apply Cutout²⁹ with a patch size of 4×4 . We apply the following methods during hardware-aware finetuning of the model. After every update, we clip the weights of each layer to a constant value ($\alpha = 1$) around zero. The tiling of layers is simulated during training with a tile size of 256×256 . In order to make the model more robust to noise, we perturb the weights of each tile according to a model simulating two times the PCM programming noise derived experimentally in Ref. 30. Additionally, we add additive Gaussian noise on the output of each per-tile MVM with standard deviation of 10% of the maximum output range. Finally, DACs and ADCs at the output of each tile are simulated during training with a resolution of 8 bits.

PTB-char LSTM network training.

The Penn Treebank (PTB)²³ corpus is a collection of articles that is commonly used for evaluating sequence labelling approaches, as well as the language modelling capabilities of models on a per-word or per-character (PTB-char) level. We use PTB-char which has a vocabulary size of 50, i.e. there are 50 different characters in the dataset. During preprocessing, every character is embedded to a random orthogonal vector (dimension $E = 128$) drawn from a standard Gaussian distribution. The training sequence of length 5,017,482 is split up into smaller sequences of length 150, which are shuffled after each epoch. The validation sequence is split up into smaller sequences of size 128, which we then use to determine the best model during training. The best model is finally evaluated on the whole test sequence of length 442,423. Since we are considering a language modelling task, the target sequence corresponds to the input sequence shifted to the left by one character.

We use a one-layer LSTM³¹, where the input gate projects the embedded inputs for each time-step to the LSTM cell (dimension $G = 2016$). The projected input is then added to the equally shaped projection of the hidden state ($H = 504$). The LSTM cell then processes the combined input and produces the next cell- and hidden state. The latter is used as input to the output layer, which applies dropout³² ($p = 0.25$) and maps the input to the output dimension, which is equal to the number of characters in the vocabulary. For the embedding, we use vectors sampled from a standard Gaussian that are orthogonalized using Gram-Schmidt orthogonalization.

The training procedure is split into the standard training phase, followed by the hardware-aware finetuning. In the standard training phase, we train the model for $N = 200$ epochs using the Adam³³ optimizer with an initial learning rate of $\eta = 0.01$, which is reduced by a multiplicative factor of $\gamma = 0.1$ every 50 epochs. We use a batch size of $B = 128$ and clip the gradients to have a maximum l_2 -norm of 5.0 in order to avoid training instabilities. After the standard training phase, we set the dropout probability to $p = 0.0$ and inject Gaussian noise into the weights during each forward pass². Given the weight matrix W , each weight $W_{i,j}$ follows the distribution $\mathcal{N}(W_{i,j}, \zeta \cdot |W_{\max} - W_{\min}|)$, where $\zeta = 0.075$ controls the amount of noise relative to the dynamic range of the weights. In order to avoid weight outliers, we clip the weights of each layer to $\alpha = 2.0$ times the layers' standard deviation. We find that for randomly distributed time-steps, the inputs to the hidden gate contain performance-critical outliers that can not be clipped post-training. We therefore also clip the inputs to the hidden gate (which are also the outputs of the LSTM cell) to a constant value of $\beta = 0.06$.

389 LSTM network for image caption generation training.

390 We use Flickr8k²⁴, which is a collection of images from Flickr with 5 human-generated captions per image. The dataset
 391 is split into a training, validation, and test set with 6k, 1k, and 1k samples, respectively. Due to size limitations incurred by
 392 the hardware, we reduce the vocabulary size from the original 8,918 to 4,064 (16 cores with output size 254) by removing the
 393 least frequently used words. As a consequence, 12% of the available captions are eliminated. Nine images are removed from the
 394 dataset altogether because every caption contains an eliminated word. Using BLEU-4, which is the most common BLEU variant,
 395 we verified that removing 12% of the captions did not significantly alter the performance of the network ($0.1609 \rightarrow 0.1557$ for
 396 the whole and reduced dataset, respectively). As a final preprocessing step, we apply basic tokenization on the sequences.

397 Similar to³⁴, we extract 2,048 features from each image using an InceptionV3³⁵ model that was pretrained on ImageNet³⁶.
 398 The extracted features are then projected to a 504-dimensional subspace, followed by batch normalization³⁷. As a generative
 399 model, we use a one-layer LSTM³¹ architecture (dimension $G = 2,016$). At time $t = 0$, the LSTM receives as input the embedded
 400 image features and an all-zero hidden state vector. At time $t > 0$, the LSTM receives as input the embedded word (learnable
 401 embedding vectors initialized with uniform distribution) and hidden state that was generated in the previous timestep. At each
 402 timestep, the output is generated by passing the 504-dimensional hidden state of the LSTM through a dense layer, producing an
 403 output vector of shape 4,064, over which a softmax is computed. At each timestep, the word with the highest softmax probability
 404 is chosen. The LSTM runs for at most 24 timesteps or until an end-of-sequence token is generated.

405 We first train a network without any hardware-aware training for $N = 70$ epochs using a batch size of $B = 120$. We use the
 406 Adam³³ optimizer with an initial learning rate of $\eta = 0.001$. To avoid overfitting, we apply dropout to the activations of the input
 407 layer with a dropout probability of $p = 0.5$. We then finetune the pretrained model for another 50 epochs using hardware-aware
 408 training. During this phase, we inject Gaussian noise (the same type as for PTB-Char, $\zeta = 0.08$) and evaluate with slightly
 409 less noise ($\zeta = 0.035$). After every update, we clip the weights to $\alpha = 2.0$ standard deviations and apply l_2 -regularization
 410 with $\lambda = 1e-4$. To avoid exploding gradients, we clip the gradient l_2 -norm to 0.5. During training, we maximize the sum of
 411 the negative log likelihood of the correct word at each step. As evaluation metric, we use the BLEU²⁷ score, which measures
 412 the precision of word n -grams between a generated and multiple reference captions. We report the n -gram BLEU score for
 413 $n \in \{1, 2, 3, 4\}$.

414 Weight mapping onto Hermes cores and functional modeling.

415 The mapping of weights onto Hermes is done by converting a weight matrix for each layer into 256x256 conductance sub-
 416 matrices, one for each core the layer is mapped onto. The weight matrix is split into the smallest possible amount of sub-matrices
 417 which have the same number of rows and columns, both less than 256. For example, the 2016x224 weight matrix of a deep layer
 418 of ResNet-9 is converted into 8 252x224 sub-matrices. Each sub-matrix is then reshaped into a 256x256 matrix by zero-filling.
 419 Finally, the weight values of each sub-matrix are converted to conductance values using Eq. (1), with $G_{\max}$ being determined on
 420 a per-core basis. The $G_{\max}$ value of a core is set to the minimum value between the maximum conductance supported by the
 421 unit-cell (80 ADC counts for ODP and 160 ADC counts for TDP, as determined in Fig. 2a) and the maximum conductance that
 422 leads to currents of all BLs to be less than the maximum current supported by the ADC (approximately $100 \mu A$). For higher
 423 BL currents, the ADC response starts to saturate, which introduces nonlinearity in the resulting MVMs and therefore must be
 424 avoided. This additional constraint means that for some layers the full unit-cell conductance range cannot be utilized. For ODP,
 425 the $G_{\max}$ value used for all networks is 80 except for the image captioning network, where the hidden gate layer uses 70 and
 426 the output layer values between 40 and 80 depending on the core. For ResNet-9 with TDP, we use $G_{\max}$ of 160 for conv0 and
 427 dense layers, values between 80 and 120 for conv1-4 and values between 120 and 140 for conv5-7. For PTB-char with TDP, we
 428 use $G_{\max}$ of 100 for the LSTM unit and 120 for the output layer. Therefore, only ResNet-9 takes significant advantage of TDP,
 429 because the lower utilization of the cores (especially for conv0 and dense layers) allows to use larger $G_{\max}$ values that improve
 430 the MVM accuracy.

431 Given the conductance mapping provided by the above procedure, we built a functional model of Hermes that includes the
 432 quantization from the digital peripheral circuits and the hardware-measured weight noise. The Hermes quantization model
 433 includes the quantization resulting from all data conversions performed by the Hermes chip during inference execution. It
 434 includes the 8-bit input quantization from PWM, 12-bit quantization from ADC, 8-bit data conversion from LDPU and a GDPU
 435 functional model (including the discrete tanh LUTs and 8-bit output data conversion). The weight noise model includes the per-
 436 core weight error measured when performing MVMs with the network weights being programmed. We perform multiple MVMs
 437 on each core and compute the resulting weight error $\text{std}(W - \hat{W})/W_{\max}$, where $\text{std}(W)$ is the standard deviation computed over
 438 all elements of W , as shown in Fig. 2c. We then fit the experimental weight error data points as a function of target weight with
 439 a polynomial function for each core. Weight noise during inference is simulated by injecting Gaussian noise to the synaptic
 440 weights with a weight-dependent standard deviation given by this polynomial function.

441 **Power measurements.**

442 We measured the Hermes chip energy efficiency and latency for three different use-cases. The first one is the chip performing
443 only MVMs at full utilization (all 64 cores active in parallel), which provides the maximal MVM efficiency of the chip that can
444 be compared with other works. The reported MVM energy includes the PCM crossbar array, ADCs, and PWMs (operations
445 performed by the LDPU and GDPU used in neural network executions, such as activation functions, are not included in the
446 MVM energy). The second use-case is the processing of one input to a deep layer of ResNet-9 that spans 8 cores, including
447 the MVMs, data aggregation with on-chip links and LDPU operations. The third use-case is the processing of one timestep of
448 the LSTM unit of the image caption generation network, including the MVMs, data aggregation with on-chip links, LDPU and
449 GDPU operations.

450 For each use-case, the power consumed by the Hermes chip is measured both in standby mode (static) and while performing
451 the computations (dynamic) to calculate the energy. The average current drawn by the ADC and digital supplies operating at
452 0.85 V is measured using voltmeters (Fluke models 87V and 185). The dynamic power consumption is obtained for all the
453 monitored supplies by sending a stream of MVM commands with a period of $5\mu\text{s}$, and measuring the averaged power during
454 continuous operation of a few seconds. The dynamic energy is computed by multiplying the dynamic power by the MVM
455 command period. The total energy is calculated by adding the static and dynamic contributions for all monitored supplies. The
456 latency is computed by running the register transfer level (RTL) code for each use-case.

457 REFERENCES

- 458 ¹ P. Yao, H. Wu, B. Gao, J. Tang, Q. Zhang, W. Zhang, J. J. Yang, and H. Qian, “Fully hardware-implemented memristor convolutional neural network,” *Nature*
459 **577**, 641–646 (2020).
- 460 ² V. Joshi, M. Le Gallo, S. Haefeli, I. Boybat, S. R. Nandakumar, C. Piveteau, M. Dazzi, B. Rajendran, A. Sebastian, and E. Eleftheriou, “Accurate deep neural
461 network inference using computational phase-change memory,” *Nature Communications* **11**, 2473 (2020).
- 462 ³ A. Biswas and A. P. Chandrakasan, “CONV-SRAM: an energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural
463 networks,” *IEEE Journal of Solid-State Circuits* **54**, 217–230 (2019).
- 464 ⁴ F. Merrikh-Bayat, X. Guo, M. Klachko, M. Prezioso, K. K. Likharev, and D. B. Strukov, “High-performance mixed-signal neurocomputing with nanoscale
465 floating-gate memory cell arrays,” *IEEE Transactions on Neural Networks and Learning Systems* **29**, 4782–4790 (2018).
- 466 ⁵ F. Cai, J. M. Correll, S. H. Lee, Y. Lim, V. Bothra, Z. Zhang, M. P. Flynn, and W. D. Lu, “A fully integrated reprogrammable memristor-CMOS system for
467 efficient multiply-accumulate operations,” *Nature Electronics* **2**, 290–299 (2019).
- 468 ⁶ W.-H. Chen, C. Dou, K.-X. Li, W.-Y. Lin, P.-Y. Li, J.-H. Huang, J.-H. Wang, W.-C. Wei, C.-X. Xue, Y.-C. Chiu, *et al.*, “CMOS-integrated memristive
469 non-volatile computing-in-memory for AI edge processors,” *Nature Electronics* **2**, 420–428 (2019).
- 470 ⁷ R. Khaddam-Aljameh, M. Stanisavljevic, J. Fornt Mas, G. Karunaratne, M. Brändli, F. Liu, A. Singh, S. M. Müller, U. Egger, A. Petropoulos, T. Anton-
471 akopoulos, K. Brew, S. Choi, I. Ok, F. L. Lie, N. Saulnier, V. Chan, I. Ahsan, V. Narayanan, S. R. Nandakumar, M. Le Gallo, P. A. Francese, A. Sebastian, and
472 E. Eleftheriou, “HERMES-Core—A 1.59-TOPS/mm² PCM on 14-nm CMOS In-Memory Compute Core Using 300-ps/LSB Linearized CCO-Based ADCs,”
473 *IEEE Journal of Solid-State Circuits* **57**, 1027–1038 (2022).
- 474 ⁸ P. Deaville, B. Zhang, L.-Y. Chen, and N. Verma, “A maximally row-parallel mram in-memory-computing macro addressing readout circuit sensitivity and
475 area,” in *ESSDERC 2021 - IEEE 51st European Solid-State Device Research Conference (ESSDERC)* (2021) pp. 75–78.
- 476 ⁹ W.-S. Khwa, Y.-C. Chiu, C.-J. Jhang, S.-P. Huang, C.-Y. Lee, T.-H. Wen, F.-C. Chang, S.-M. Yu, T.-Y. Lee, and M.-F. Chang, “A 40-nm, 2m-cell, 8b-precision,
477 hybrid slc-mlc pcm computing-in-memory macro with 20.5 - 65.0tops/w for tiny-al edge devices,” in *2022 IEEE International Solid- State Circuits Conference*
478 *(ISSCC)*, Vol. 65 (2022) pp. 1–3.
- 479 ¹⁰ W. Wan, R. Kubendran, C. Schaefer, S. B. Eryilmaz, W. Zhang, D. Wu, S. Deiss, P. Raina, H. Qian, B. Gao, *et al.*, “A compute-in-memory chip based on
480 resistive random-access memory,” *Nature* **608**, 504–512 (2022).
- 481 ¹¹ J.-M. Hung, C.-X. Xue, H.-Y. Kao, Y.-H. Huang, F.-C. Chang, S.-P. Huang, T.-W. Liu, C.-J. Jhang, C.-I. Su, W.-S. Khwa, *et al.*, “A four-megabit compute-in-
482 memory macro with eight-bit precision based on cmos and resistive random-access memory for ai edge devices,” *Nature Electronics* **4**, 921–930 (2021).
- 483 ¹² L. Fick, S. Skrzyniarz, M. Parikh, M. B. Henry, and D. Fick, “Analog matrix processor for edge ai real-time video analytics,” in *2022 IEEE International*
484 *Solid- State Circuits Conference (ISSCC)*, Vol. 65 (2022) pp. 260–262.
- 485 ¹³ H. Jia, M. Ozatay, Y. Tang, H. Valavi, R. Pathak, J. Lee, and N. Verma, “Scalable and programmable neural network inference accelerator based on in-memory
486 computing,” *IEEE Journal of Solid-State Circuits* **57**, 198–211 (2022).
- 487 ¹⁴ P. Narayanan, S. Ambrogio, A. Okazaki, K. Hosokawa, H. Tsai, A. Nomura, T. Yasuda, C. Mackin, S. C. Lewis, A. Friz, M. Ishii, Y. Kohda, H. Mori, K. Spoon,
488 R. Khaddam-Aljameh, N. Saulnier, M. Bergendahl, J. Demarest, K. W. Brew, V. Chan, S. Choi, I. Ok, I. Ahsan, F. L. Lie, W. Haensch, V. Narayanan, and G. W.
489 Burr, “Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format,” *IEEE*
490 *Transactions on Electron Devices* **68**, 6629–6636 (2021).
- 491 ¹⁵ B. Murmann, “Mixed-signal computing for deep neural network inference,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* **29**, 3–13
492 (2021).
- 493 ¹⁶ M. Dazzi, A. Sebastian, T. Parnell, P. A. Francese, L. Benini, and E. Eleftheriou, “Efficient pipelined execution of CNNs based on in-memory computing and
494 graph homomorphism verification,” *IEEE Transactions on Computers* **70**, 922–935 (2021).
- 495 ¹⁷ I. Boybat, M. Le Gallo, S. Nandakumar, T. Moraitis, T. Parnell, T. Tuma, B. Rajendran, Y. Leblebici, A. Sebastian, and E. Eleftheriou, “Neuromorphic
496 computing with multi-memristive synapses,” *Nature communications* **9**, 2514 (2018).
- 497 ¹⁸ R. Khaddam-Aljameh, M. Martemucci, B. Kersting, M. Le Gallo, R. L. Bruce, M. BrightSky, and A. Sebastian, “A Multi-Memristive Unit-Cell Array With
498 Diagonal Interconnects for In-Memory Computing,” *IEEE Transactions on Circuits and Systems II: Express Briefs* **68**, 3522–3526 (2021).
- 499 ¹⁹ N. Papandreou, H. Pozidis, A. Pantazi, A. Sebastian, M. Breitwisch, C. Lam, and E. Eleftheriou, “Programming algorithms for multilevel phase-change
500 memory,” in *IEEE International Symposium on Circuits and Systems (ISCAS)* (IEEE, 2011) pp. 329–332.
- 501 ²⁰ M. Le Gallo, S. Nandakumar, L. Ciric, I. Boybat, R. Khaddam-Aljameh, C. Mackin, and A. Sebastian, “Precision of bit slicing with in-memory computing
502 based on analog phase-change memory crossbars,” *Neuromorphic Computing and Engineering* **2**, 014009 (2022).
- 503 ²¹ M. Le Gallo, A. Sebastian, G. Cherubini, H. Giefers, and E. Eleftheriou, “Compressed sensing with approximate message passing using in-memory comput-
504 ing,” *IEEE Transactions on Electron Devices* **65**, 4304–4312 (2018).
- 505 ²² A. Krizhevsky, “Learning multiple layers of features from tiny images,” *Tech. Rep.* (2009).
- 506 ²³ M. P. Marcus, B. Santorini, and M. A. Marcinkiewicz, “Building a large annotated corpus of English: The Penn Treebank,” *Computational Linguistics* **19**,
507 313–330 (1993).
- 508 ²⁴ C. Rashtchian, P. Young, M. Hodosh, and J. Hockenmaier, “Collecting image annotations using amazon’s mechanical turk,” in *Proceedings of the NAACL*
509 *HLT 2010 Workshop on Creating Speech and Language Data with Amazon’s Mechanical Turk*, CSLDAMT ’10 (Association for Computational Linguistics,
510 USA, 2010) p. 139–147.
- 511 ²⁵ M. J. Rasch, D. Moreda, T. Gokmen, M. Le Gallo, F. Carta, C. Goldberg, K. El Maghraoui, A. Sebastian, and V. Narayanan, “A flexible and fast pytorch
512 toolkit for simulating training and inference on analog crossbar arrays,” in *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and*
513 *Systems (AICAS)* (2021) pp. 1–4.
- 514 ²⁶ A. Mujika, F. Meier, and A. Steger, “Fast-slow recurrent neural networks,” in *Advances in Neural Information Processing Systems*, Vol. 30, edited by I. Guyon,
515 U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Curran Associates, Inc., 2017).
- 516 ²⁷ K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: A method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting*
517 *on Association for Computational Linguistics*, ACL ’02 (Association for Computational Linguistics, USA, 2002) p. 311–318.
- 518 ²⁸ K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition*
519 *(CVPR)* (IEEE Computer Society, Los Alamitos, CA, USA, 2016) pp. 770–778.
- 520 ²⁹ T. Devries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout,” *CoRR* **abs/1708.04552** (2017), arXiv:1708.04552.
- 521 ³⁰ S. R. Nandakumar, I. Boybat, V. Joshi, C. Piveteau, M. Le Gallo, B. Rajendran, A. Sebastian, and E. Eleftheriou, “Phase-change memory models for deep
522 learning training and inference,” in *2019 26th IEEE International Conference on Electronics, Circuits and Systems (ICECS)* (2019) pp. 727–730.
- 523 ³¹ S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.* **9**, 1735–1780 (1997).

- ⁵²⁴ ³²N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach.*
⁵²⁵ *Learn. Res.* **15**, 1929–1958 (2014).
- ⁵²⁶ ³³D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations, ICLR 2015, San Diego,*
⁵²⁷ *CA, USA, May 7-9, 2015, Conference Track Proceedings*, edited by Y. Bengio and Y. LeCun (2015).
- ⁵²⁸ ³⁴O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, “Show and tell: A neural image caption generator,” in *2015 IEEE Conference on Computer Vision and Pattern*
⁵²⁹ *Recognition (CVPR)* (2015) pp. 3156–3164.
- ⁵³⁰ ³⁵C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *2016 IEEE Conference on*
⁵³¹ *Computer Vision and Pattern Recognition (CVPR)* (2016) pp. 2818–2826.
- ⁵³² ³⁶O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “Imagenet large
⁵³³ scale visual recognition challenge,” *Int. J. Comput. Vision* **115**, 211–252 (2015).
- ⁵³⁴ ³⁷S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd Interna-*
⁵³⁵ *tional Conference on International Conference on Machine Learning - Volume 37, ICML’15 (JMLR.org, 2015)* p. 448–456.

⁵³⁶ ACKNOWLEDGMENTS

⁵³⁷ We thank Geoffrey W. Burr, Silvia M. Mueller, Yasuteru Kohda, Kohji Hosakawa, Fee Li Lie, Feng Liu, Ted Levin and Tarl
⁵³⁸ Gordon for assistance with the chip design; Jordi Fornt Mas and Giorgio Cristiano for chip testing and simulation; Frédéric
⁵³⁹ Odermatt, Irem Boybat, S. R. Nandakumar, Christophe Piveteau, Corey Lammie, Hadjer Benmeziane and Malte Rasch for help
⁵⁴⁰ with network deployment on the chip; Angeliki Pantazi, Robert Haas, Alessandro Curioni, Sidney Tsai, Wilfried Haensch, Jeff
⁵⁴¹ Burns, Rama Divakaruni and Mukesh Khare for managerial support. We would also like to thank Prof. Luca Benini and Prof.
⁵⁴² Bipin Rajendran for their support with supervising students. This work was supported by the IBM Research AI Hardware Center.
⁵⁴³ We also acknowledge partial funding from the European Research Council (ERC) under the European Union’s Horizon 2020
⁵⁴⁴ research and innovation programme (grant agreement numbers 682675 and 966764).

⁵⁴⁵ AUTHOR CONTRIBUTIONS

⁵⁴⁶ COMPETING INTERESTS

⁵⁴⁷ The authors declare no competing interests.

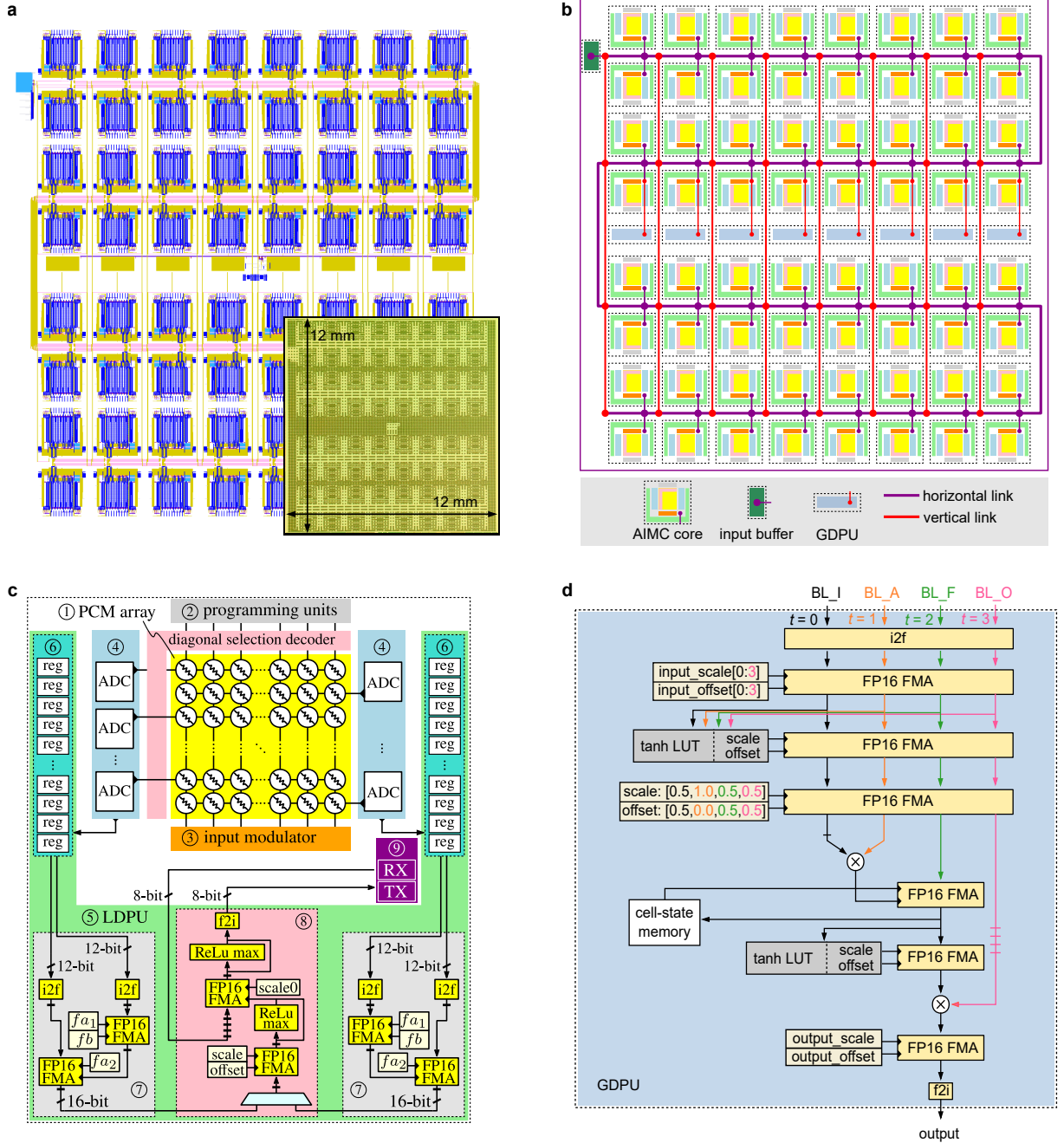


FIG. 1. **Hermes chip overview.** **a**, EDA snapshot and inset showing a micrograph of the chip. Therein, the outline of the 64 cores can be recognized as well as the array of 5,616 pads. **b**, Schematic overview of the different components on the multi-core Hermes chip. **c**, Schematic overview of a single PCM-based in-memory compute core. (1) PCM crossbar array, (2) current DAC-based programming unit, (3) PWM-based input modulator, (4) left and right ADC arrays, (5) local digital processing unit (LDPU), (6) left and right ADC register arrays, (7) left and right ADC convert and scale blocks, (8) activation function block, (9) link controller. **d**, Block diagram of a global digital processing unit (GDPU) used for LSTM-related data processing. Inputs to and outputs from the GDPU slice are in an 8-bit signed integer format (INT8). By using custom conversion blocks marked by $i2f$ and $f2i$, INT8 values can be converted into FP16 and vice versa. Additionally, conversions at input/output can encompass a per-gate/per-output scale and bias operation using the FMA units. The inputs from the I, A, F, and O BLs are time multiplexed, and a single block is used to compute the gates' activation vectors. The sigmoid activation function for the I, F and O gates is computed by scaling and offsetting the output of the hyperbolic tangent function with the third (from the top) FMA unit, according to the identity $\text{sigmoid}(x) = 1/2 + 1/2 \cdot \tanh(x/2)$.

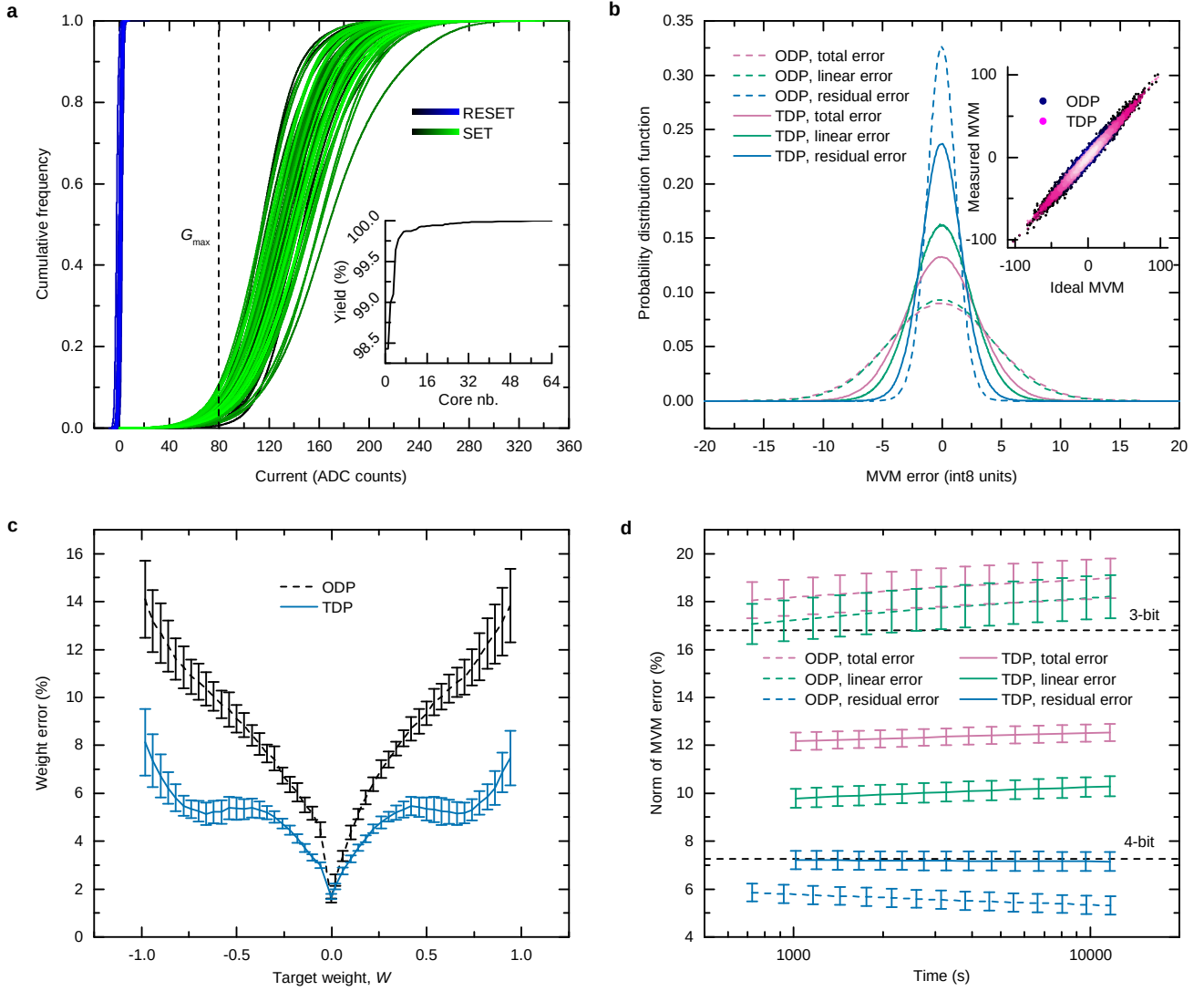


FIG. 2. Hermes MVM characterization. **a**, Unit-cell SET and RESET distributions of the 64 cores. Shades of blue and green denote different cores. The dashed line represents the ODP $G_{\max}$, defined as the tenth percentile of the core with the least conductive SET states. The inset shows the unit-cell yield of the 64 cores. The yield condition is that the unit-cell can be programmed to $|G_{\text{RESET}}| < 5$ and $G_{\text{SET}} > 50$. **b**, Error distributions of ϵ_{total} , ϵ_{linear} and $\epsilon_{\text{residual}}$ for the 64 cores in LDPU (int8) units. The reduction of ϵ_{total} achieved with TDP can be attributed to a reduction of ϵ_{linear} . The distributions are computed over all error vector elements of 2,048 MVMs performed with all 64 cores. The inset shows the measured MVM results of one core for ODP and TDP against the ideal MVMs computed in software. **c**, Weight error $\text{std}(W - \hat{W})/W_{\max}$, where $\text{std}(W)$ is the standard deviation computed over all elements of W , as a function of target weight for ODP and TDP. The error bars represent one standard deviation over the 64 cores. **d**, 2-norm of ϵ_{total} , ϵ_{linear} and $\epsilon_{\text{residual}}$, normalized by $\|y_{\text{fp}}\|_2$, as a function of time for ODP and TDP. Due to temporal conductance drift of PCM devices, ϵ_{total} and ϵ_{linear} increase gradually. The dashed lines represent the error achieved by a digital engine with 8-bit input/output precision and N -bit weight precision. The error bars represent one standard deviation over the 64 cores.

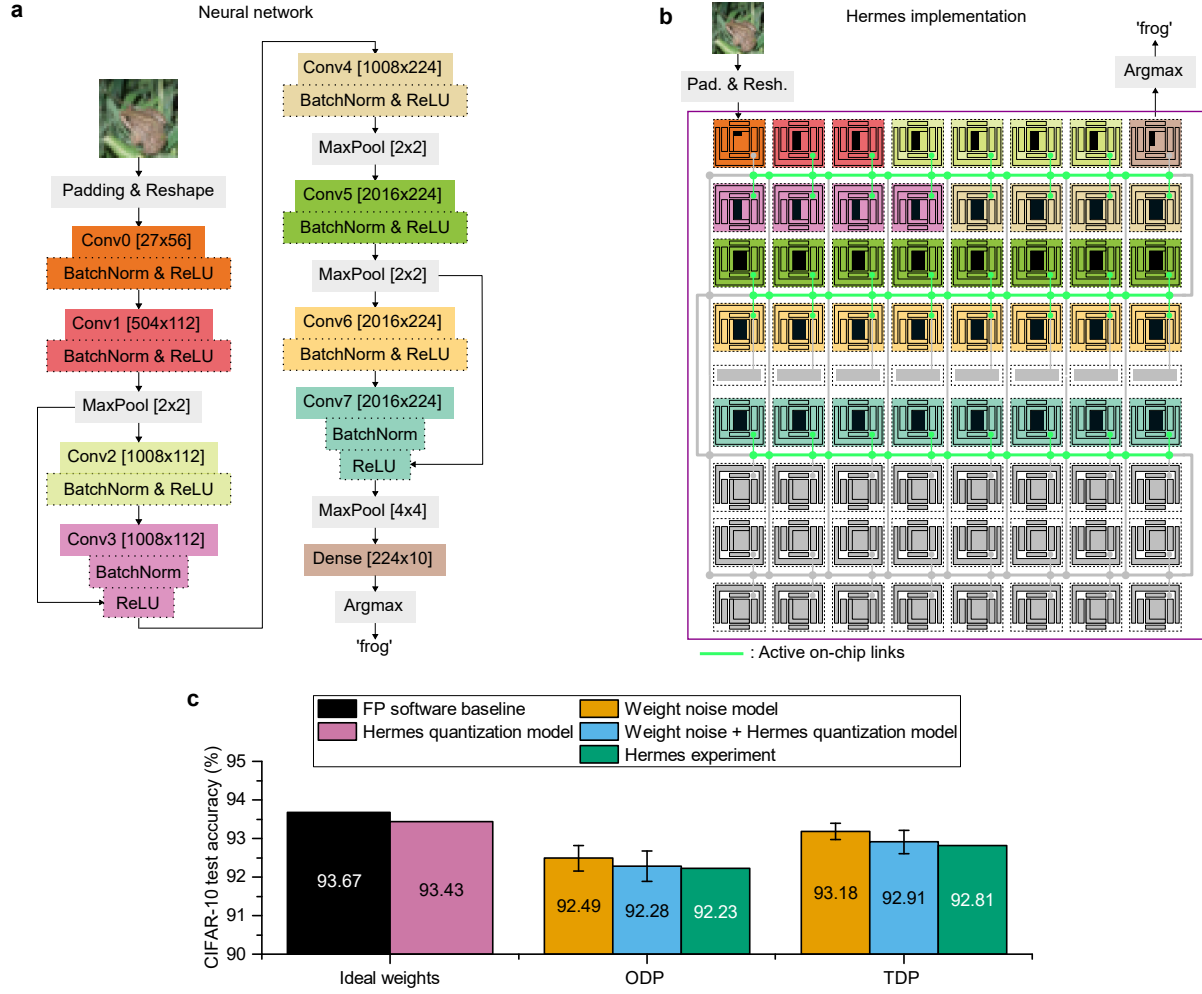


FIG. 3. ResNet-9 on CIFAR-10 measurement results. **a**, Network architecture. ResNet-9 comprises 8 convolution layers and 1 dense classification layer. Each convolution layer is followed by batch normalization and ReLU activation. Two residual connections from Conv1 to Conv3 and Conv5 to conv7, are present. Four max-pooling layers, implemented off-chip, are used to reduce the size of the input volume along network depth. **b**, Mapping of ResNet-9 onto the Hermes chip. Weights of all layers are programmed onto 40 cores. Conv0-4 and the dense layer are implemented on the first two rows of cores, whereas Conv5-7 each span an entire row. On-chip links connect the LDPU's of the cores within a layer to realize on-chip data aggregation. Batch normalization, ReLU, residual and bias additions are implemented in the LDPU's. **c**, Measured test accuracy results on CIFAR-10 benchmark for ODP and TDP compared with software baseline and simulation results that include the hardware-measured weight noise and quantization from the PWM and LDPU. The error bars represent one standard deviation over 10 inference runs.

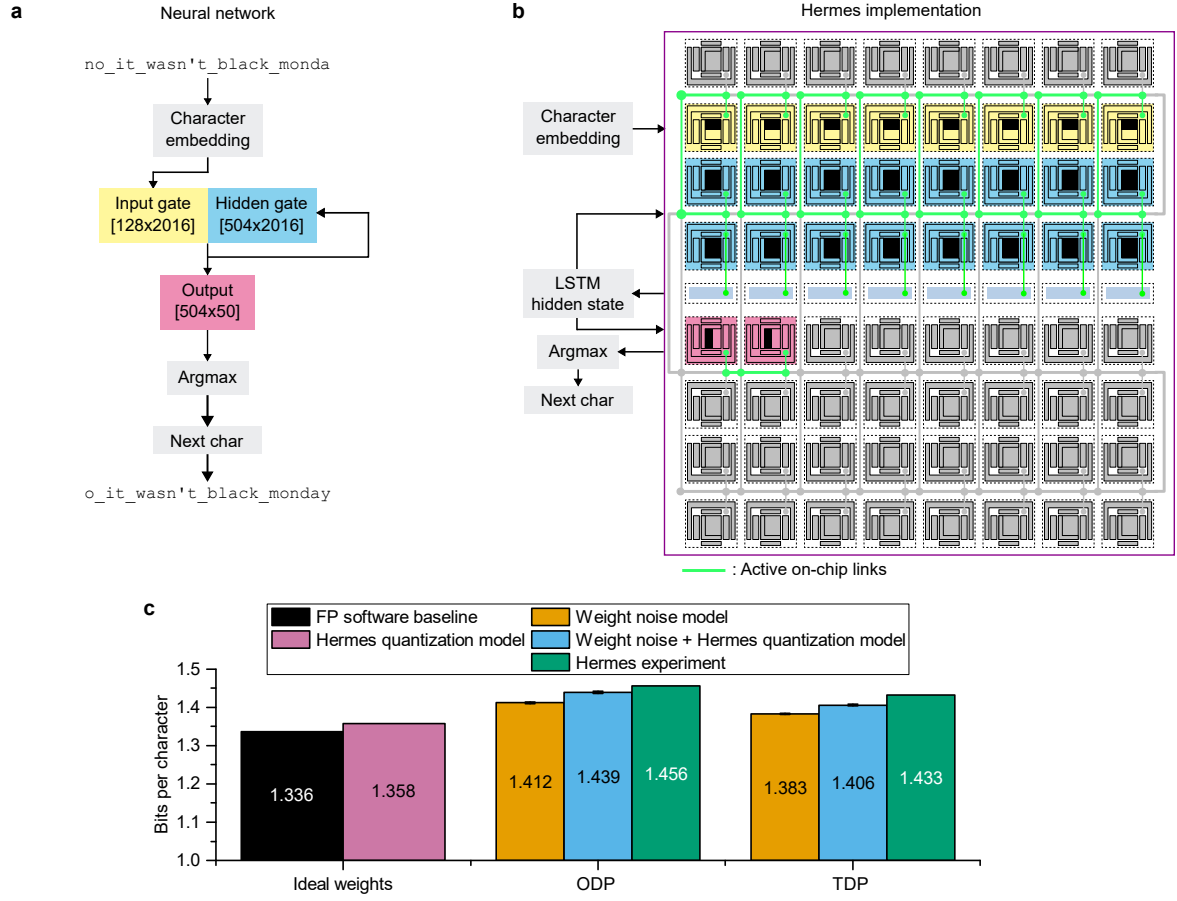


FIG. 4. LSTM for character prediction measurement results. **a**, Network architecture. The LSTM unit comprises an input gate that receives the embedded input characters and a hidden gate whose input is the output hidden state vector of the LSTM unit. The output layer is a dense classification layer whose output is the next predicted character. **b**, Mapping of the LSTM network onto the Hermes chip. Weights of all layers are programmed onto 26 cores of the chip. The rows 2, 3 and 4 encode the weights LSTM unit and vertical on-chip links connect their LDPU to realize on-chip data aggregation. The LDPU of the cores in row 4 are connected to their respective GDPUs via on-chip links. **c**, Measured BPC results for ODP and TDP compared with software baseline and simulation results that include the hardware-measured weight noise and quantization from the PWM, LDPU, and GDPUs. The error bars represent one standard deviation over 10 inference runs.

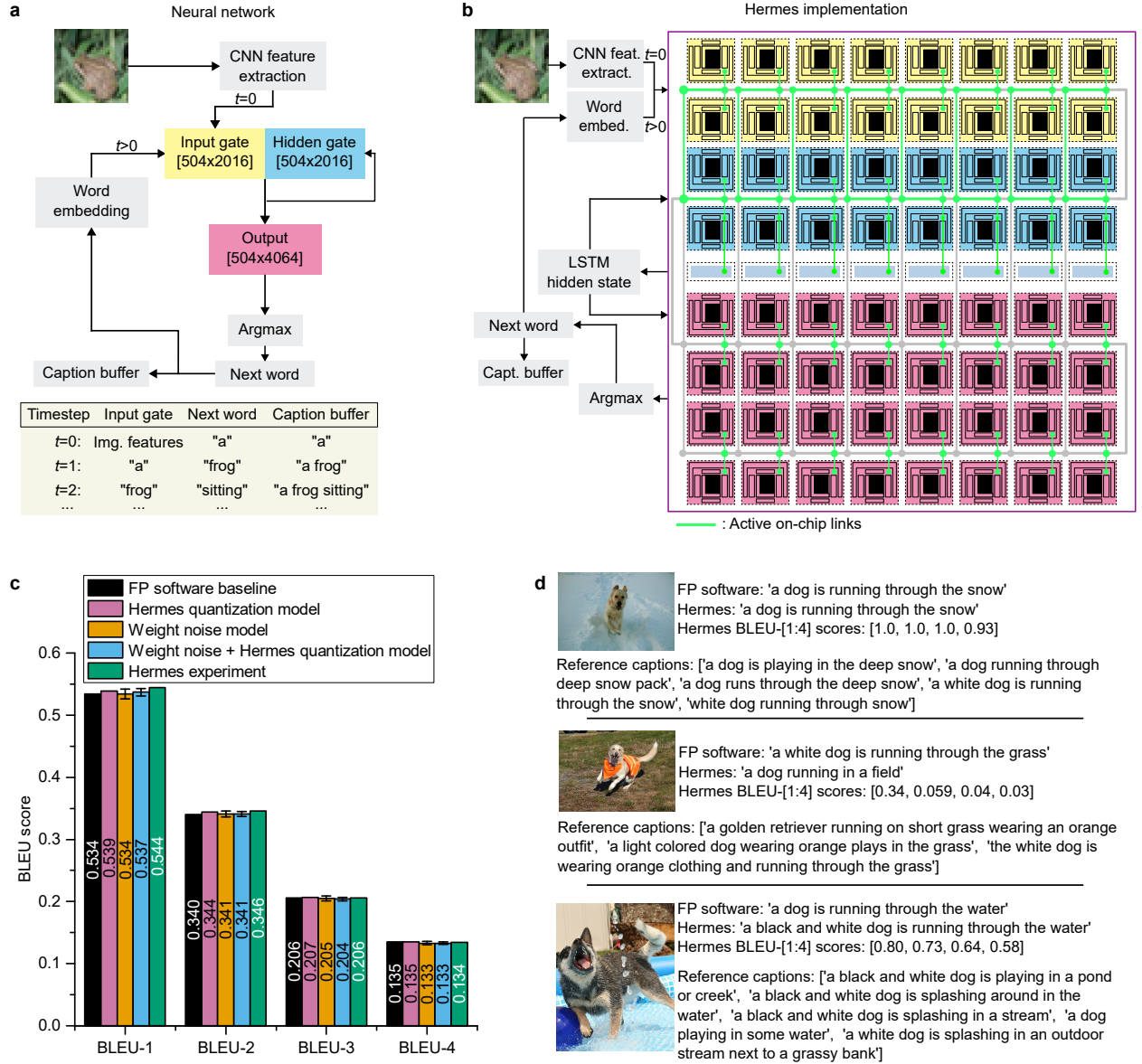


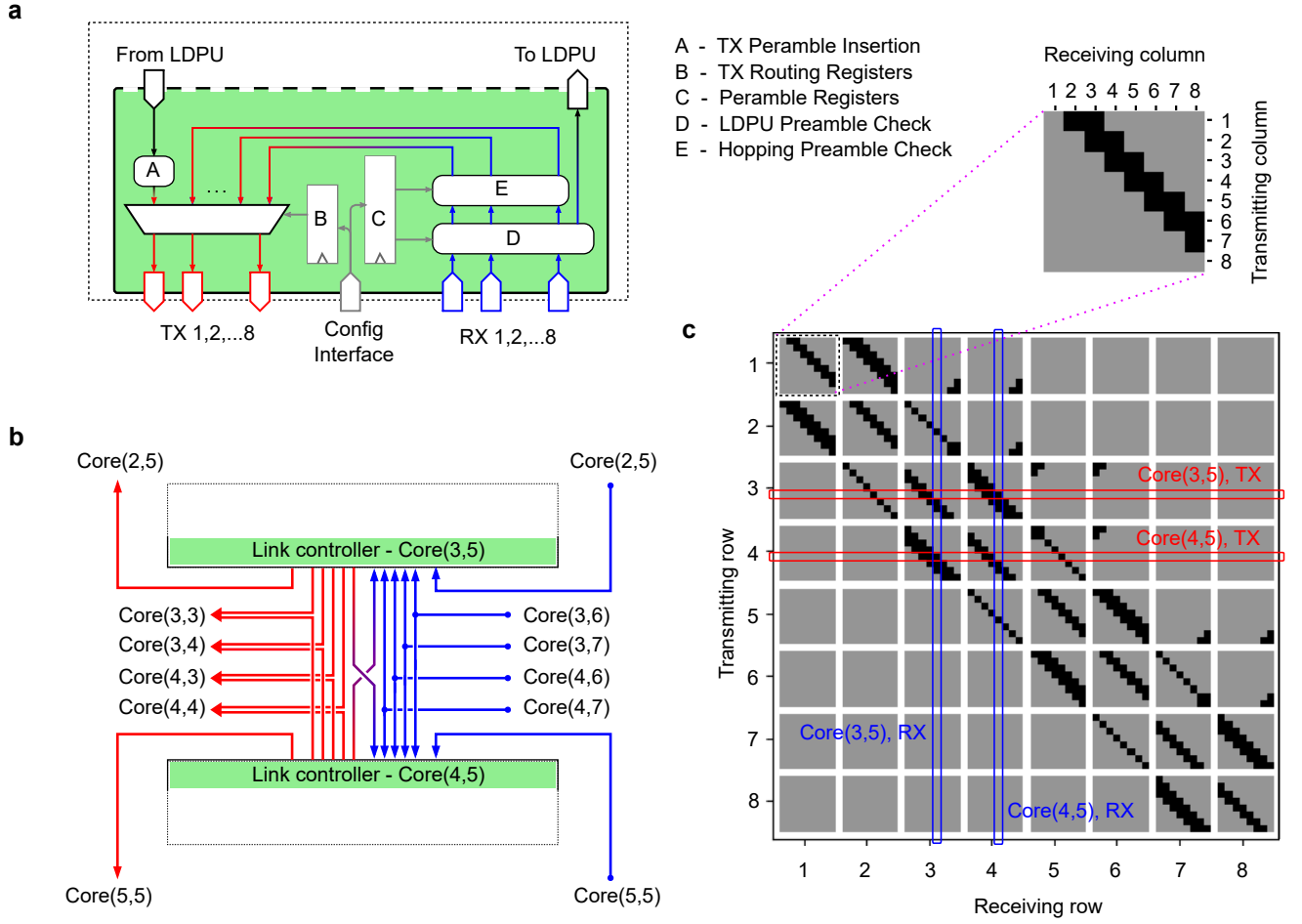
FIG. 5. LSTM for image caption generation measurement results. **a**, Network architecture. The features of the input image extracted by a CNN (off-chip) are fed to the input gate of the LSTM unit. The output hidden state vector of the LSTM unit is sent to the output dense layer that generates the first word of the caption, which is stored in a buffer. This word is then embedded and fed back to the input gate of the LSTM unit to generate the next word. **b**, Mapping of the network onto the Hermes chip. Weights of LSTM and dense layers are programmed onto all 64 cores. The rows 1-4 encode the weights LSTM unit and vertical on-chip links connect their LDPU slices to realize on-chip data aggregation. The LDPU slices of the cores in row 4 are connected to their respective GDPU slice via on-chip links. The dense layer is mapped on rows 5-8, and on-chip vertical data aggregation is setup within rows 5-6 and rows 7-8. **c**, Measured BLEU- n scores for Hermes with ODP compared with software baseline and simulation results that include the hardware-measured weight noise and quantization from the PWM, LDPU, and GDPU. Only ODP was used for this network because $G_{\max}$ cannot be increased beyond 80 ADC counts to stay within the linear region of the ADC, therefore TDP does not bring accuracy benefits (see Methods). The error bars represent one standard deviation over 10 inference runs. **d**, Comparison between software and Hermes generated captions for three exemplary images, along with the reference captions over which the BLEU scores are computed.

	MVM at max. utilization		ResNet-9 layer		One timestep of LSTM unit	
Number of cores used	64		8		32	
Average core utilization	100%		86%		97%	
Read mode ^a	1-phase	4-phase	1-phase	4-phase	1-phase	4-phase
Peak MVM throughput (TOPS) ^b	63.1	16.1	6.79	1.74	30.6	7.82
Peak MVM energy efficiency (TOPS/W) ^b	9.76	2.48	6.88	1.74	9.34	2.37
MVM area efficiency (TOPS/mm ²) ^b	1.55	0.40	1.34	0.34	1.50	0.38
Energy (μ J)	0.86	3.38	0.97	1.51	3.46	5.24
Latency (ns)	133	520	1131	1518	1043	1430

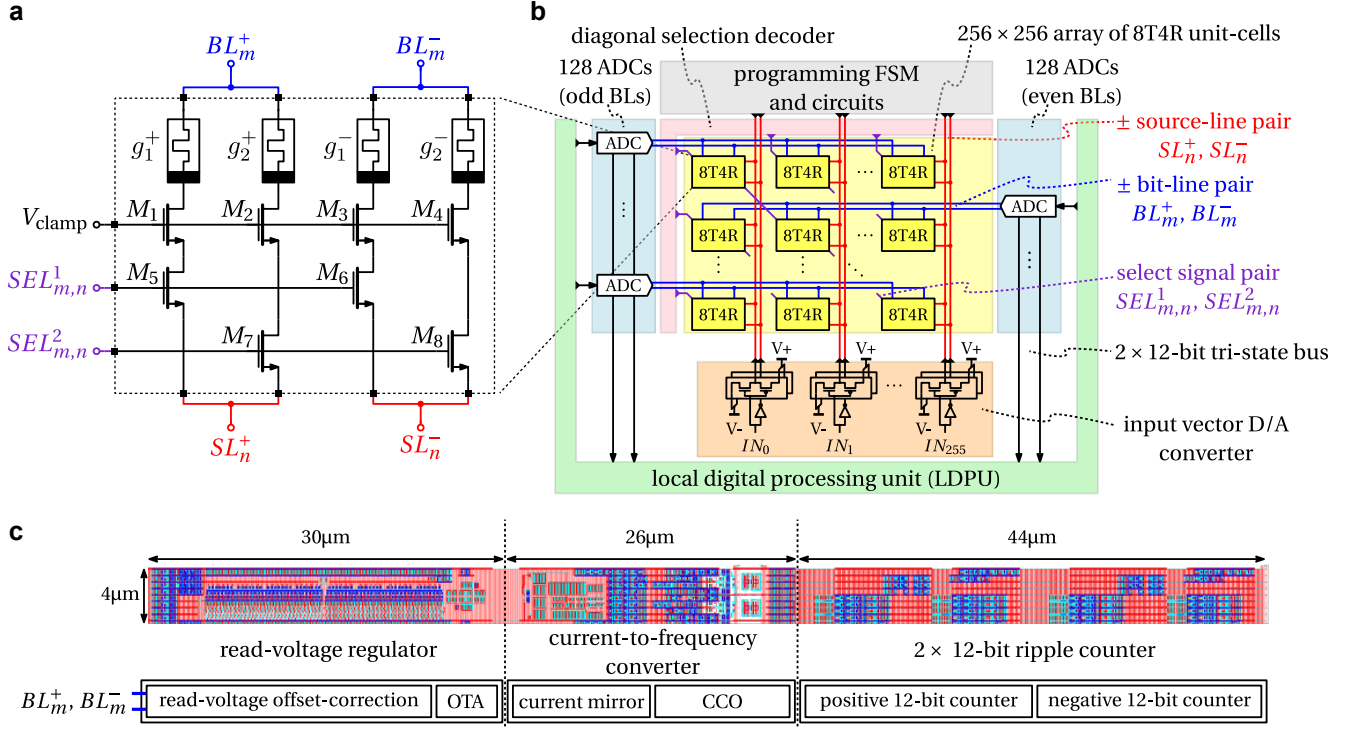
^a All MVM and inference accuracies are reported for 4-phase mode.

^b Throughput, energy, and area consider only the MVM operations performed by the Hermes cores. 1 multiply-and-accumulate = 2 OPs.

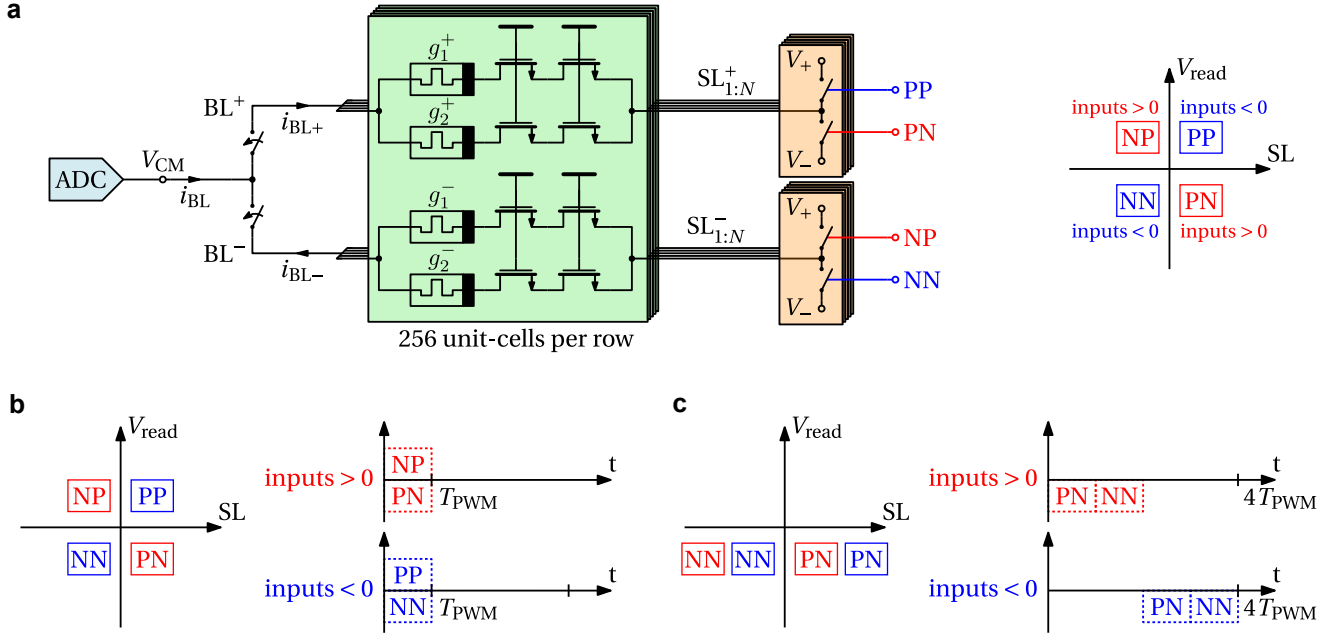
TABLE I. Measured speed and energy efficiency of Hermes chip.



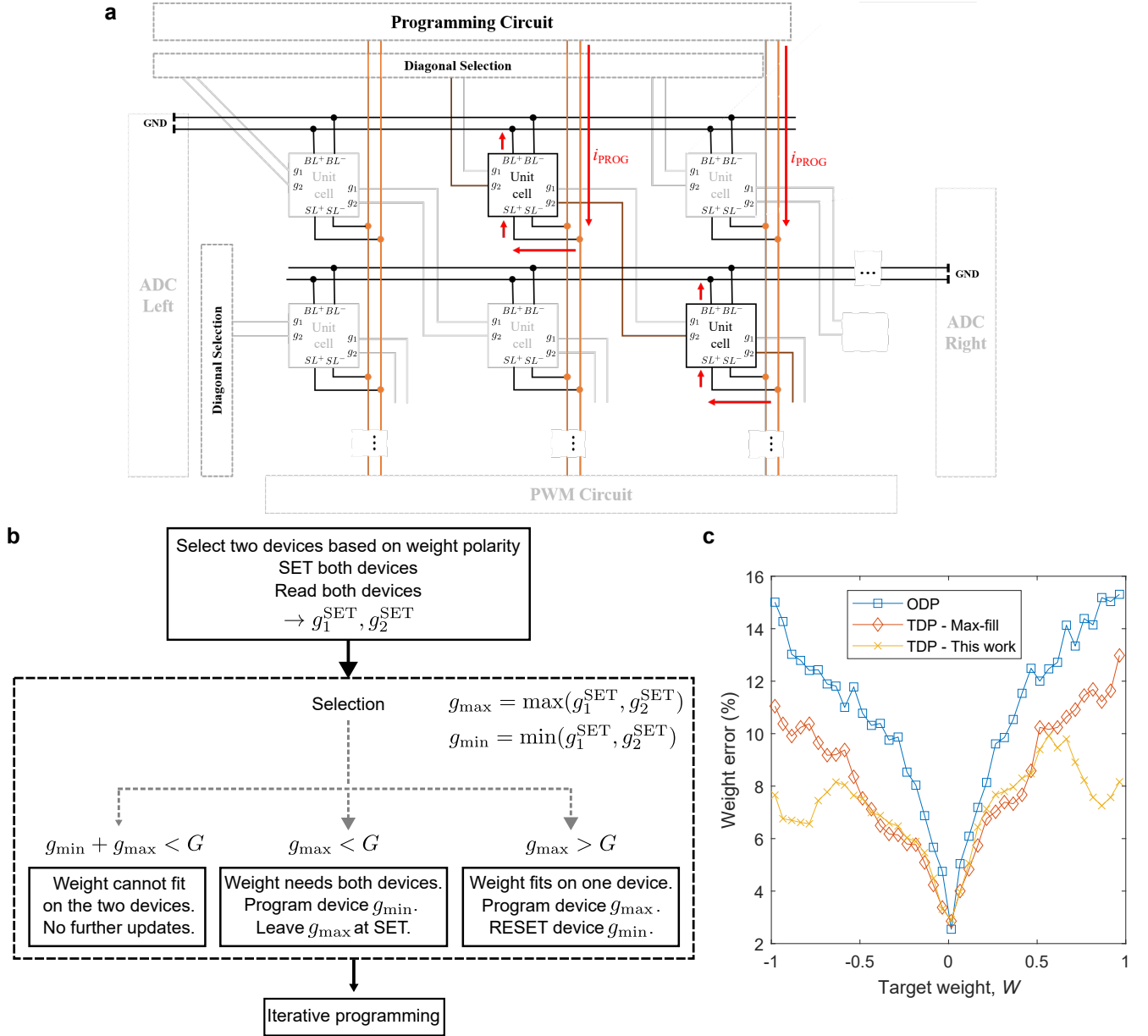
EXTENDED DATA FIG. 1. **Digital communication fabric.** **a**, Schematic of link controller. **b**, Possible link connections for Core(3,5) and Core(4,5), where the notation Core(r,c) refers to the core located at row r and column c in Fig. 1b. **c**, Link connections for the entire Hermes chip. The RX and TX connections for Core(3,5) and Core(4,5) shown in **b** are indicated.



EXTENDED DATA FIG. 2. PCM crossbar array. **a**, Schematic of 8T4R unit-cell. The top electrodes of the conductance pairs of each polarity connect to separate bit lines BL_m^+ , BL_m^- and the sources of their lower access-transistors connect to separate source lines SL_n^+ , SL_n^- . Thus, the devices in a conductance pair are weighted with equal significance and the total conductance per unit-cell becomes: $(g_1^+ + g_2^+) - (g_1^- + g_2^-)$. **b**, Schematic of PCM crossbar array. To program the PCM devices, the dedicated per-core programming FSM instructs the diagonal selection decoder to enable one diagonal of cells that contains the devices that are to be programmed. The diagonal selection decoder controls the $SEL_{m,n}^1$ and $SEL_{m,n}^2$ signals in the unit-cell, which are routed diagonally throughout the array. The selected devices are programmed by the current-steering DAC-based programming units located on top of the PCM array. To perform an MVM, the 256 inputs to the crossbar array ($IN_0 - IN_{255}$) are applied via the red source lines (SLs) to the 8T4R cells. The resulting bit line (BL) currents are summed up on the blue wires and read by the ADCs that flank the crossbar array on the left and right. **c**, Layout of one ADC. The block diagram that is shown below the layout illustrates the various components of the ADC, namely, the read voltage regulator, the current-to-frequency converter, and the 2 × 12-bit ripple counter.



EXTENDED DATA FIG. 3. **Input modulation modes for MVM.** **a**, Full array read procedure for MVMs showing the connection between ADC, unit-cells, and input modulator switches. Signals PP and PN connect the positive source lines $SL_{1:N}^+$ to the positive potential V^+ and V^- , respectively. For NP and NN it is vice versa. **b**, 1-phase modulation mode. Inputs of positive and negative polarity are applied to weights of positive and negative polarity in one modulation cycle T_{PWM} . **c**, 4-phase modulation mode. Inputs of positive and negative polarity are applied individually to weights of positive and negative polarity in four modulation cycles.



EXTENDED DATA FIG. 4. **Weight programming procedure.** **a**, Crossbar array during programming. **b**, Proposed TDP algorithm to program a target conductance value G on a unit-cell. **c**, Weight error comparison between TDP of this work and previous approaches. TDP - Max-fill refers to programming the two devices with iterative programming up to the ODP $G_{\max}$, as proposed in Ref. 20. Due to the wide SET distribution shown in Fig. 2a, some devices in the core either cannot achieve $G_{\max}$, or conversely could be programmed to much higher conductance values than $G_{\max}$. Therefore, the latter approach leads to programming inaccuracies resulting from either under-utilizing the conductance range of individual devices or from devices that cannot reach $G_{\max}$. The proposed TDP algorithm solves this issue by using the readout SET conductance of the devices of the unit-cell to map the weight.

Name	Value
Technology	14-nm CMOS + IBM PCM
Supply voltage	0.85 – 0.95 V
Chip Area	12 mm × 12 mm = 144 mm ²
Single core area	1.2 mm × 1.16 mm = 1.392 mm ²
MVM area per core	0.870 mm × 0.730 mm = 0.635 mm ²
GDPUs macro area	0.809 mm × 0.347 mm = 0.281 mm ²
Number of cores	64
Number of GDPUs	8
Number of C4 pads per core	72
Total number of C4 pads	5,616
External contact pads	1,520
Maximum MVM clock frequency	1 GHz
Maximum LDPU/GDPU clock frequency	400 MHz
Maximum link data-rate	3.2 kBit/s
Number of unit-cells per core	256 × 256 = 65,536
Number of PCM devices per unit cell	4
Number of PCM devices per core	256 × 256 × 4 = 262,144
Number of PCM devices per chip	256 × 256 × 4 × 64 = 16,777,216
Nominal PCM resistance range per PCM device	100 kΩ – 10 MΩ
Resolution of the time-encoded inputs	8 bit (1 ns/LSB)
Read voltage range	100 – 400 mV
Number of ADCs per core	256 (1 per row)
ADC output bits	2 × 12 bit
Number of write heads per core	32
Max. supported write current per write-head	800 μA

EXTENDED DATA TABLE I. Summary of Hermes chip specifications.

	Hermes (this work)		ISSCC '22 [9]	Nature '22 [10]	Nat. Elec. '21 [11]	ISSCC '22 [12]	JSSC '22 [13]
	1-phase read	4-phase read					
CMOS technology	14 nm		40nm	130 nm	22 nm	40 nm	16nm
AIMC device	PCM		PCM	RRAM	RRAM	nor-Flash	SRAM
Chip area (mm ²)	144		18	159	6	190	25
Number of cores (core size)	64 (256x256)		8 (256x1024)	48 (256x256)	8 (1024x512)	76 (1024x1024)	16 (1152x256)
Number of weights	4.2M		400k	1.6M	524k	80M	1.2M
Input/weight/output precision	8b/Analog/8b		8b/8b/19b	4b/Analog/6b	8b/8b/14b	8b/Analog/8b	4b/4b/8b
CIFAR-10 accuracy	-	92.81%	91.89%	85.66%	92.01%	-	91.51%
Peak MVM throughput (TOPS)	63.1	16.1	0.475	0.754	0.0337	16.6	11.8
MVM TOPS/W	9.76	2.48	20.5	16	15.6	5.2	121
MVM GOPS/mm ²	1550	400	26.4	4.7	5.61	-	2670
Non-MVM operations supported	ReLU/sigmoid/tanh, batch norm., LSTM hidden state, partial sum accumulation		-	ReLU/sigmoid/tanh	-	All (1 RISC-V processor per core)	All CNN/LSTM operations (SIMD ALU + LUTs)

EXTENDED DATA TABLE II. Comparison of Hermes with other multi-core AIMC chips demonstrating neural network inference.